

HW04 - AI Automation Testing Report

Student: Nguyen Phan Hung Linh Student ID: 23127081 Repository:
<https://github.com/linhnph05/hw04---Automation-testing>

1. Scope and feature selection

I tested these three web features:

- Pool A - FR-03: Forgot password and reset password.
- Pool B - FR-09: Discount coupons.
- Pool C - FR-12: Admin access control.

2. Method

I used AI step by step. First, I asked a Codex agent to read the requirements and source code. Next, I asked it to design 12 test cases for each feature. I reviewed the cases before asking for a Playwright draft for each feature. I changed the drafts from CommonJS to ES modules, fixed incorrect assumptions, and ran the tests in Chromium. I then asked the AI to review its drafts and made a final check myself.

3. Automated cases and assertion patterns

Each JSON file contains 12 test cases. There are 36 cases in total. Running them in three browsers produces 108 test executions.

All 36 cases are written in a simple text format in TEST_CASES.md (TEST_CASES.md). This file shows the setup, input, actions, and expected result. Playwright reads the test data from the JSON files.

The suite uses more than three assertion patterns, including:

1. Check visibility and item count with `toBeVisible()` and `toHaveCount()`.
2. Check exact values and status codes with `toBe()`.
3. Check text with `toMatch()` and `toContainText()`.
4. Check the current page with `toHaveURL()`.
5. Check object values with `toMatchObject()`.
6. Check rejected actions with `not.toEqual()` and confirm that blocked data was not saved.

4. Execution results

Feature	Chromium	Firefox	WebKit	Total passed	Total failed
FR-03	7 passed / 5 failed	7 / 5	7 / 5	21	15
FR-09	7 passed / 5 failed	7 / 5	7 / 5	21	15
FR-12	5 passed / 7 failed	5 / 7	5 / 7	15	21
Total	19 / 17	19 / 17	19 / 17	57	51

The results are the same in all three browsers. This means the failures are likely SUT defects, not browser problems. I opened all nine HTML reports and checked that each report shows Run by: 23127081 and an ISO timestamp.

5. Human review of AI-generated drafts

AI draft problem	My correction	Why the AI missed it
Used CommonJS require in an ES module project	Changed the drafts to import and loaded JSON from files	The AI used a common Playwright example without checking the project settings
FR-03 TC-01 checked OTP length and step controls together	TC-01 now checks the OTP only; TC-02 checks the controls	The AI combined two requirements into one case
Used exact alert text to check password reset	Accepted the alert and checked the URL and login result	Alert text can change and is not the main result
Guest coupon test matched the Login link in the menu	Checked the redirect to the Login page instead	The AI did not limit the locator to the coupon result area
Used a deleted coupon as an inactive coupon	Replaced it with a real empty-code case	The API cannot create a coupon with <code>is_active = 0</code>
Sent <code>user_id</code> in normal coupon requests	Normal requests now use the user from the JWT token	The AI copied the unsafe behavior of the current client
Checked user isolation and percentage math in one case	TC-11 checks user isolation; TC-01 checks percentage math	One case had two different reasons to fail
FR-12 tests checked only the HTTP status	Also checked that blocked products, categories, and coupons were not saved	The first draft did not check database effects
Used only <code>Date.now()</code> for fixture names	Added a counter to stop duplicate names	Tests can create two fixtures in the same millisecond

6. Defects found

I created eight GitHub issues. Each issue describes one main defect, so I did not create the same issue again for every browser:

- FR-03: four-digit OTP; missing step/navigation/confirmation controls; valid password rejected.
- FR-09: incorrect percent math; wrong equality boundary; missing authentication and trusted body user ID.
- FR-12: missing admin-role checks; unauthenticated product write APIs.

Issue links, screenshots, and test mappings are in `BUG_REPORT.md` (`BUG_REPORT.md`).

7. Report evidence

- HTML reports: `reports/html/<feature>-<browser>/index.html`.
- Visual summaries: `evidence/*-chromium-report.png`.
- Failed tests include a Playwright trace and screenshot in the HTML report.

8. Conclusion

The final test suite uses separate data files and several assertion styles. It runs in Chromium, Firefox, and WebKit and produces nine HTML reports. The tests found important calculation and access-control defects. AI helped me read the code and create the first drafts, but I still had to fix the module format, selectors, test data, assertions, and cleanup.